

1. Basic Image Recognition Based on Python (OpenCV)

```
2. Predict from Image File
```

```
3. Exit
```

```
Enter your choice (1-3): 1
```

```
🔗 Starting Interactive Drawing...
```

```
Instructions: Draw digit with mouse, SPACE to recognize, C to clear, Q to quit
```

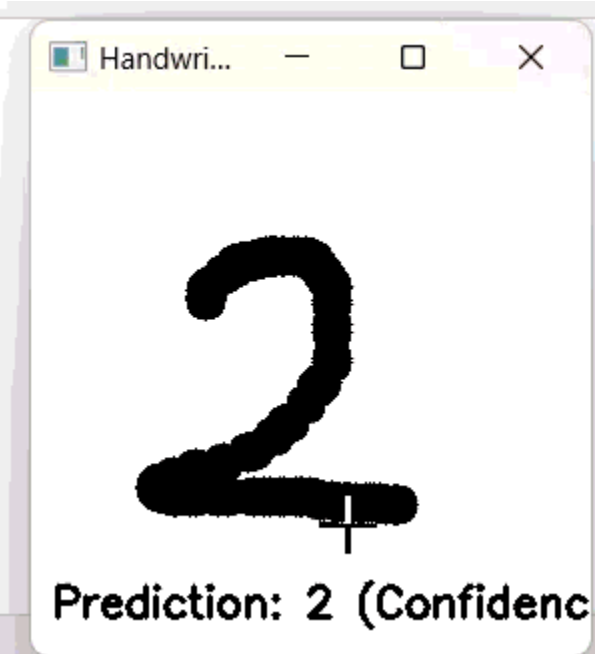
```
🔗 Prediction: 7 (Confidence: 0.469)
```

```
🔗 Prediction: 5 (Confidence: 0.309)
```

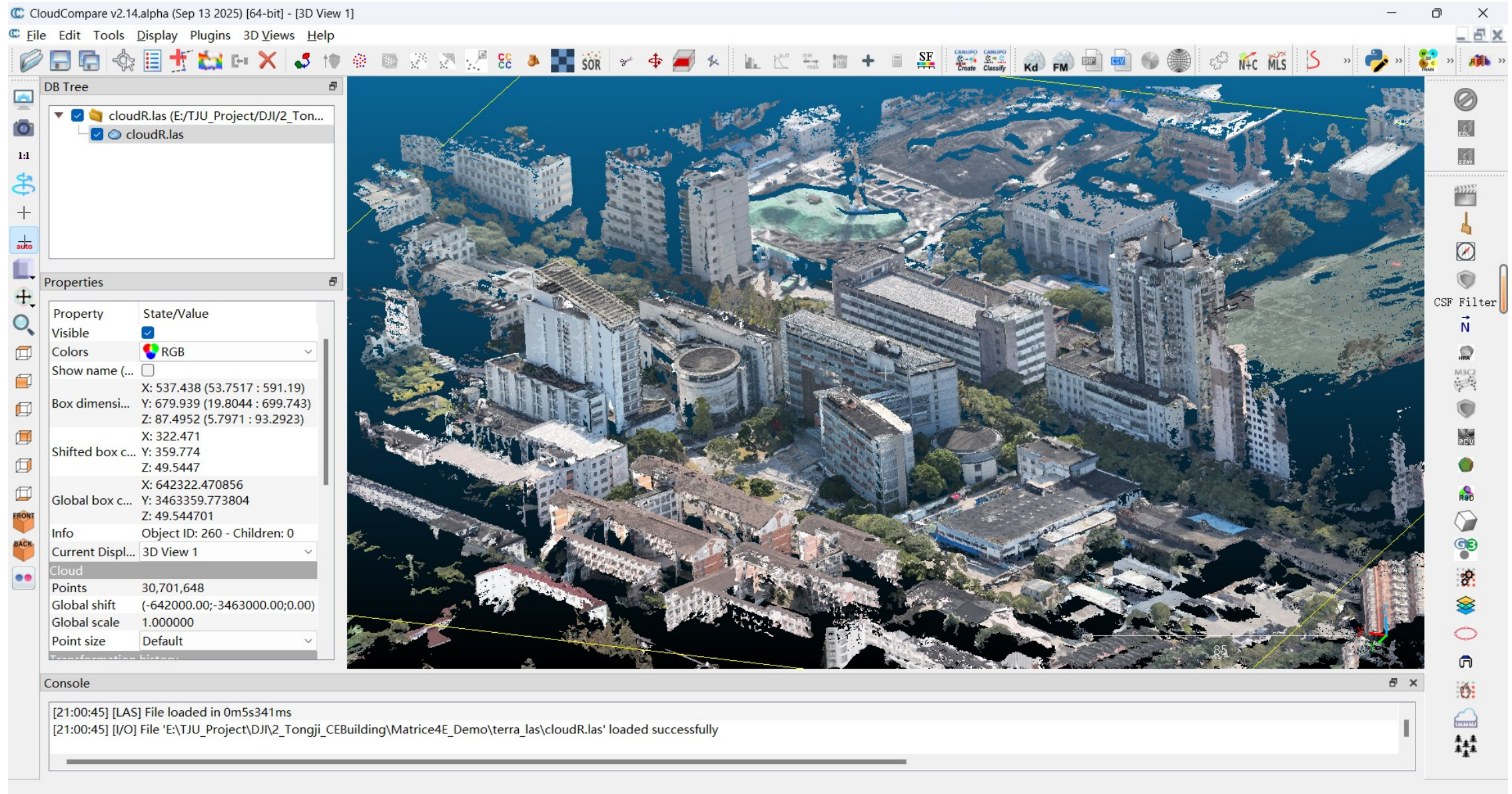
```
🔗 Prediction: 5 (Confidence: 0.316)
```

```
🔗 Prediction: 2 (Confidence: 0.659)
```

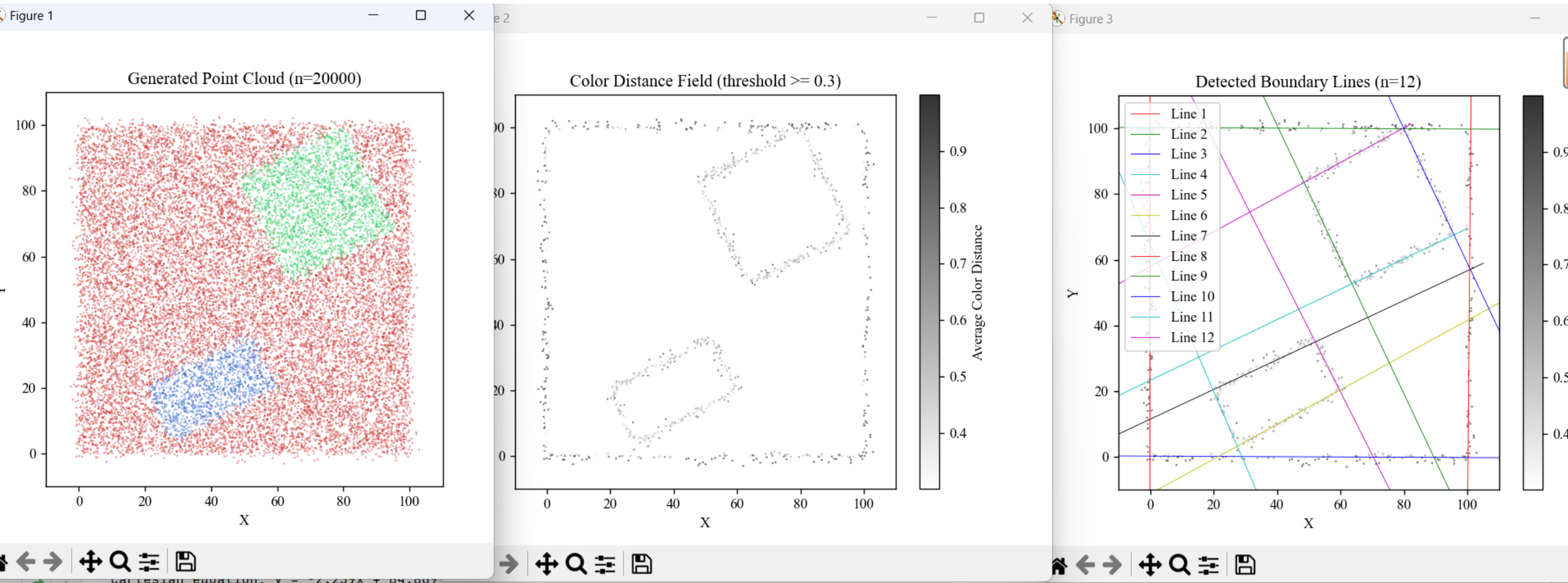
TODO Problems Terminal Python Packages Python Console



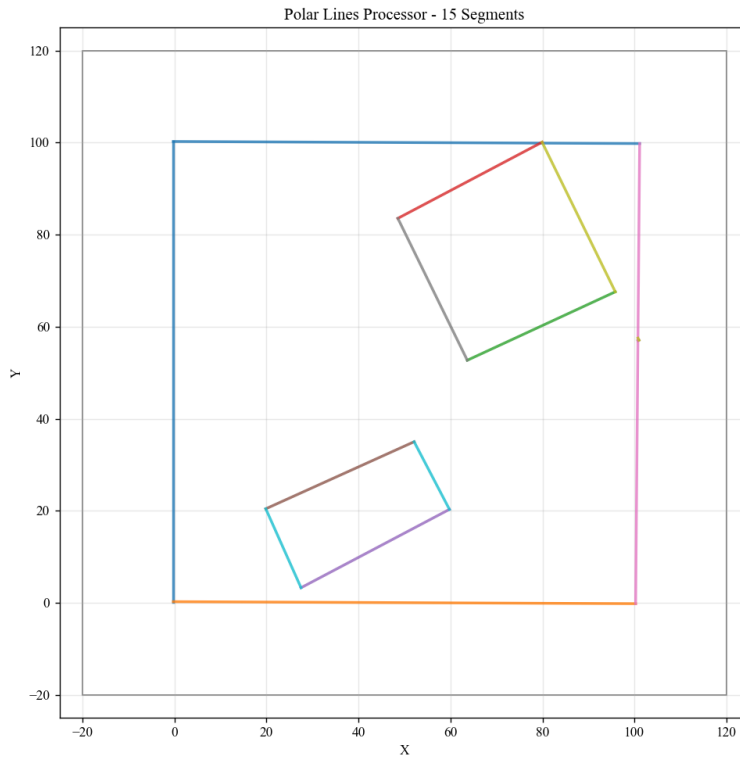
2. Large-scale 3D Reconstruction Base on Python and COLMAP



3. Demo of a Point Cloud Vectorizer I Developed – “RGB-Guided Hough Transform”



3. Demo of a Point Cloud Vectorizer I Developed – “RGB-Guided Hough Transform”



Line 1:

Polar equation: $\rho = -0.24$, $\theta = 0.00^\circ$

Cartesian equation: $x = -0.243$ (vertical line)

Line 2:

Polar equation: $\rho = 100.23$, $\theta = 89.75^\circ$

Cartesian equation: $y = -0.004x + 100.233$

Line 3:

Polar equation: $\rho = 0.24$, $\theta = 89.75^\circ$

Cartesian equation: $y = -0.004x + 0.243$

Line 4:

Polar equation: $\rho = 21.11$, $\theta = 114.82^\circ$

Cartesian equation: $y = 0.462x + 23.263$

Line 5:

Polar equation: $\rho = 51.21$, $\theta = 117.83^\circ$

Cartesian equation: $y = 0.528x + 57.905$

Line 6:

Polar equation: $\rho = -9.95$, $\theta = 117.83^\circ$

Cartesian equation: $y = 0.528x + -11.252$

Line 7:

Polar equation: $\rho = 10.44$, $\theta = 114.32^\circ$

Cartesian equation: $y = 0.452x + 11.452$

Line 8:

Polar equation: $\rho = -100.23$, $\theta = 179.50^\circ$

Cartesian equation: $y = 114.270x + -11454.036$

Line 9:

Polar equation: $\rho = 80.33$, $\theta = 26.07^\circ$

Cartesian equation: $y = -2.044x + 182.777$

Line 10:

Polar equation: $\rho = 115.76$, $\theta = 26.07^\circ$

Cartesian equation: $y = -2.044x + 263.397$

Line 11:

Polar equation: $\rho = 26.45$, $\theta = 24.07^\circ$

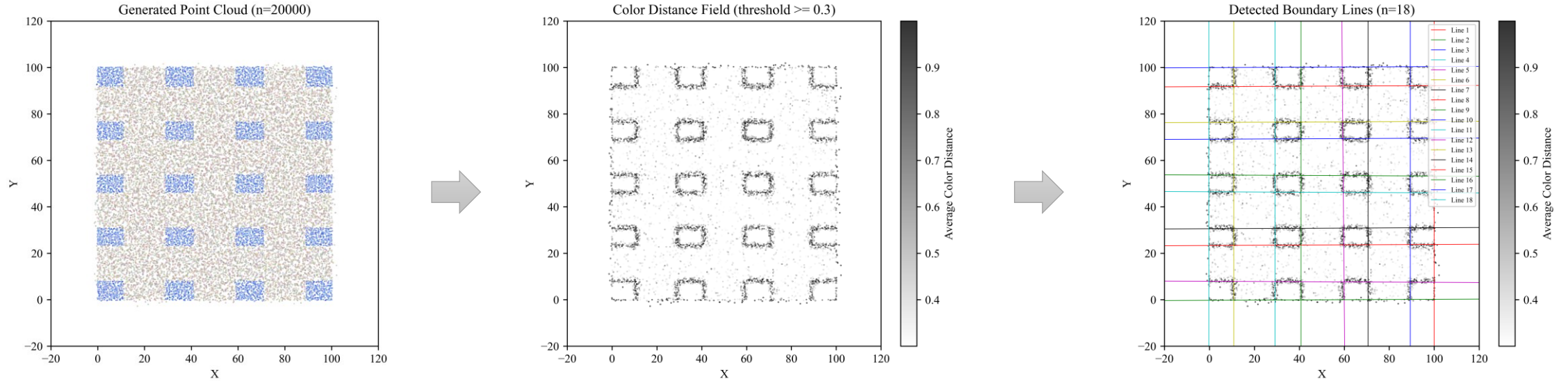
Cartesian equation: $y = -2.239x + 64.869$

Line 12:

Polar equation: $\rho = 62.37$, $\theta = 27.58^\circ$

Cartesian equation: $y = -1.915x + 134.732$

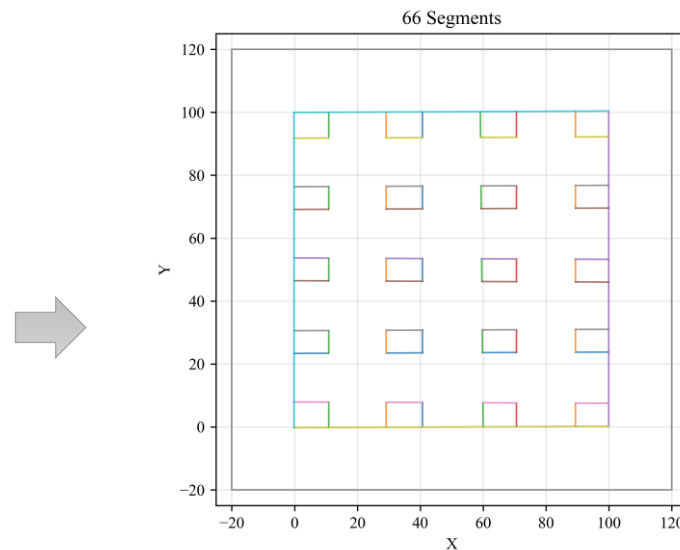
3. Demo of a Point Cloud Vectorizer I Developed – “RGB-Guided Hough Transform”



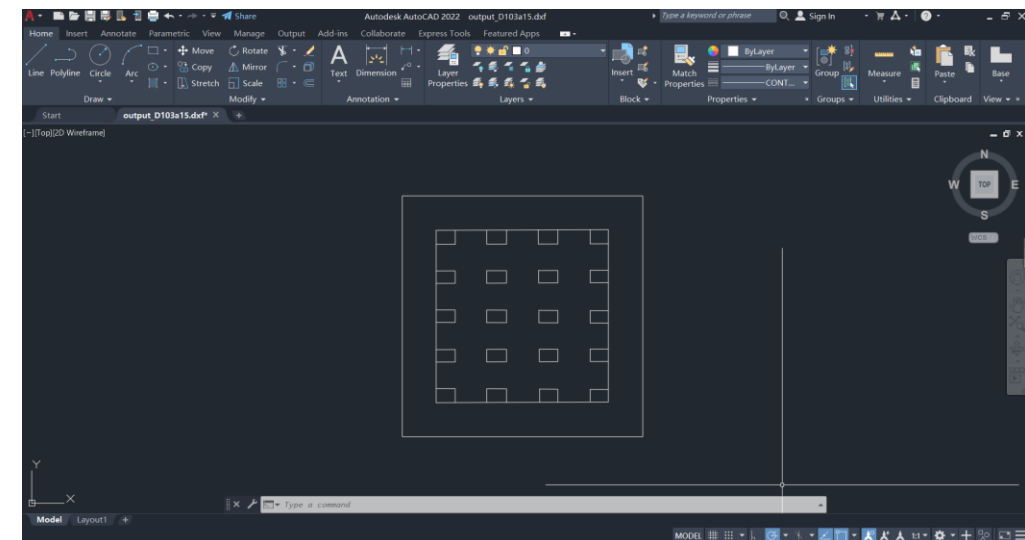
Step 1: RGB-point cloud input

Step 2: Color distance field extraction

Step 3: Hough Transform – lines detection



Step 4: Line segmentations



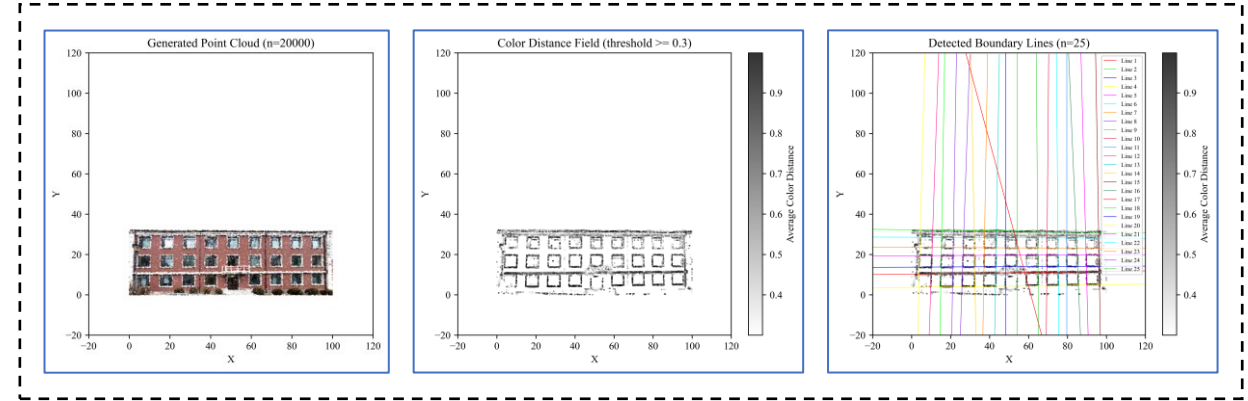
Step 5: Output to CAD

4. 3D Parametric Sementic Reconstruction Pipeline I proposed

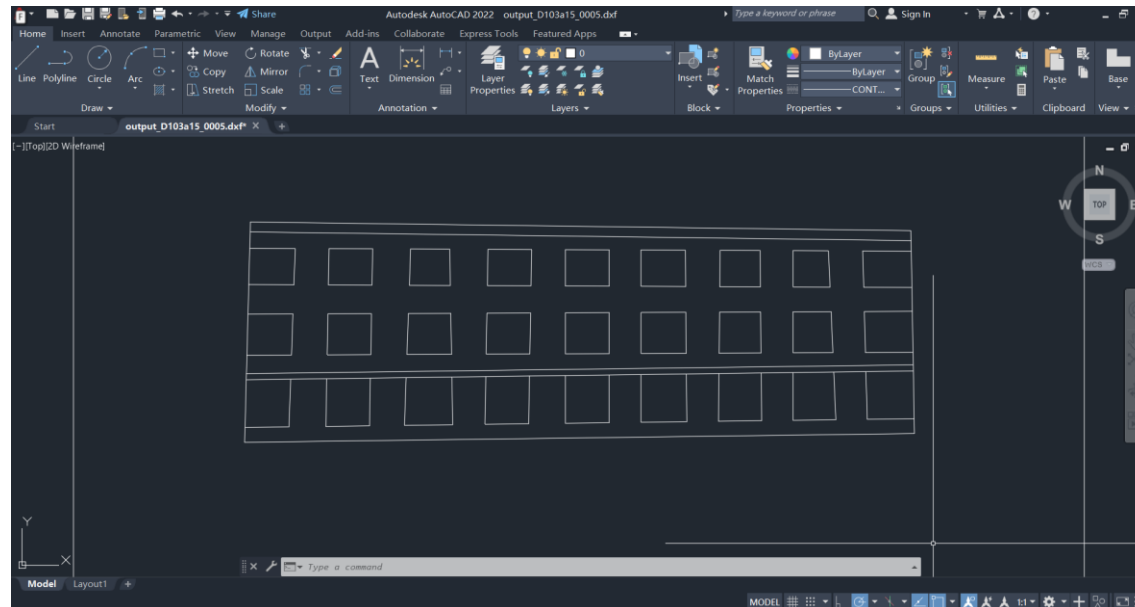


Point cloud

Phase 1: 3D reconstruction (SFM)



Phase 2: RGB-Guided Hough Transform (Automated Vectorization)



Precise Boundaries

Phase 3: Edit or polish in parametric software (e.g., Revit, CAD and Grasshopper)

CONTINUED ON NEXT PAGE

4. 3D Parametric Sementic Reconstruction Pipeline I proposed

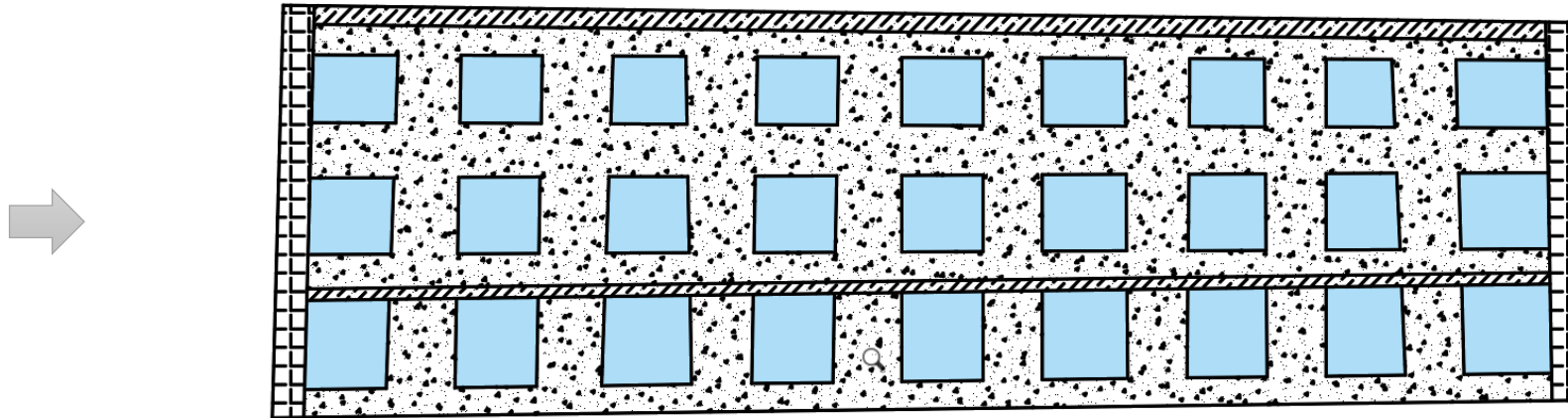
CONTINUED FROM PREVIOUS PAGE

Precise Boundaries



Phase 4: Point cloud segmentations

Phase 5: Sementic Recognition
(Maybe based on PointNet++,
Point Transformer, etc.)



Phase 6: Sementic Filling

5. Summary & Analysis

My views:

Neural networks (e.g., CNN, VLM) excel at classification (semantic recognition), but their drawback is instability. If we use them for end-to-end reconstruction, even identical training sets can yield different results on identical test sets in different training cycles. This makes the outcomes non-reproducible and unreliable as a basis for parameterization.

Color information (RGB) has been embedded in all point clouds after SFM, which is often ignored in semantic recognition. To make use of RGB, I developed the **RGB-guided Hough Transform**. I believe the key to solving this issue lies in RGB-supervised fusion with neural networks: the neural network focuses on semantic classification, while RGB conducts the numerical computations for parameterization. This ensures that results remain consistent as long as the parameters are unchanged, and the generated models are vector-based and can be imported into parametric software for analysis.

Expected Merits:

- **Process:** Segmentation-first, semantic recognition-second
- **White-Box Structure:** Consistent and reproducible results
- **Vectorized Storage:** Editable and ready for modeling analysis
- **High Robustness:** Features a unified fundamental principle, enabling versatile application across multiple scenarios.

Challenges & Potential Limitations:

1. The RGB-guided Hough Transform is susceptible to surroundings like lighting conditions. Its performance perhaps also be worse when with complex color patterns facades. How can we improve its robustness?
2. Are there scenarios where this method fails? Like in some certain situations, do end-to-end neural networks hold a greater advantage?
3. With proper optimization, does this approach have the potential to genuinely replace manual building modeling, and enable rapid digital twin for large-scale building groups?